

Glovebox

Computer-use automation for legacy back-office banking applications — design write-up

One LLM-driven run against a legacy bank UI becomes a **typed, reviewable, deterministic capability** that an AI agent invokes in production **without a model in the loop** — inside an allowlist, with redacted evidence, and with a real path for a human to take over the live session and hand it back.

The through-line

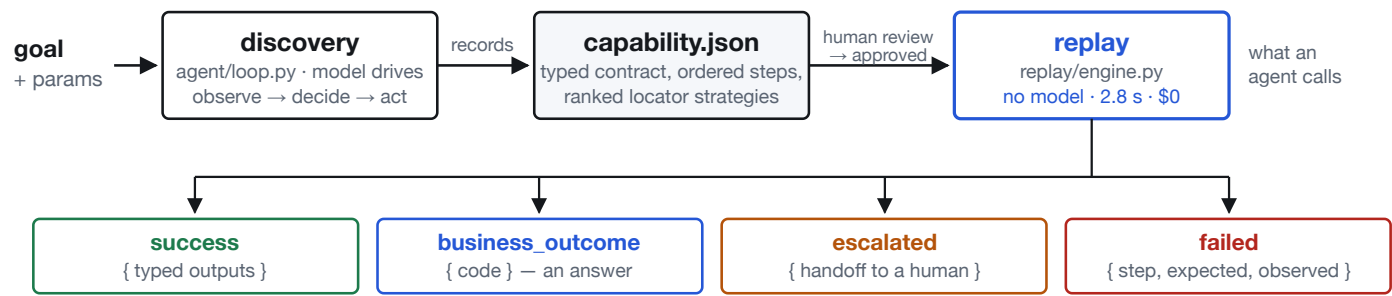


Figure 1 — The model discovers once; the artifact is what production runs. The four statuses are the contract a calling agent branches on. Separating `business_outcome` from `failed` is the load-bearing distinction: “no such member” is an answer the capability promises, not the capability breaking — conflating them is what makes a reliability number lie.

2.8 s replay through a browser, no tokens — against 86 s and ~\$0.20 for discovery	0.03 s same artifact replayed through a surface with no DOM at all	100% of 10 simulated redesigns survived, across 2 capabilities — gated in CI	253 tests (237 Python incl. real headless browser + 16 frontend)
--	--	--	--

What is load-bearing

Perception, not markup

Steps address what an *operator perceives* — role, name, label, a cell at a row and column — never what Playwright does. The target app is deliberately hostile: framesets, table layouts, `<font>` tags, no ids, no test ids, a native `confirm()`.

Every target carries its own defense

A target is an *ordered list* of independent strategies, each stored with the recorder’s note on why it should hold. Which rung survives a redesign is unknowable at record time — so all of them are kept, and drift measures which one actually caught the fall.

Discovery and replay share the same guardrail, surface and evidence code. A capability cannot do anything on replay that policy would have refused during discovery — fidelity is structural, not a promise.

1 · Architecture — one process, four layers, two seams

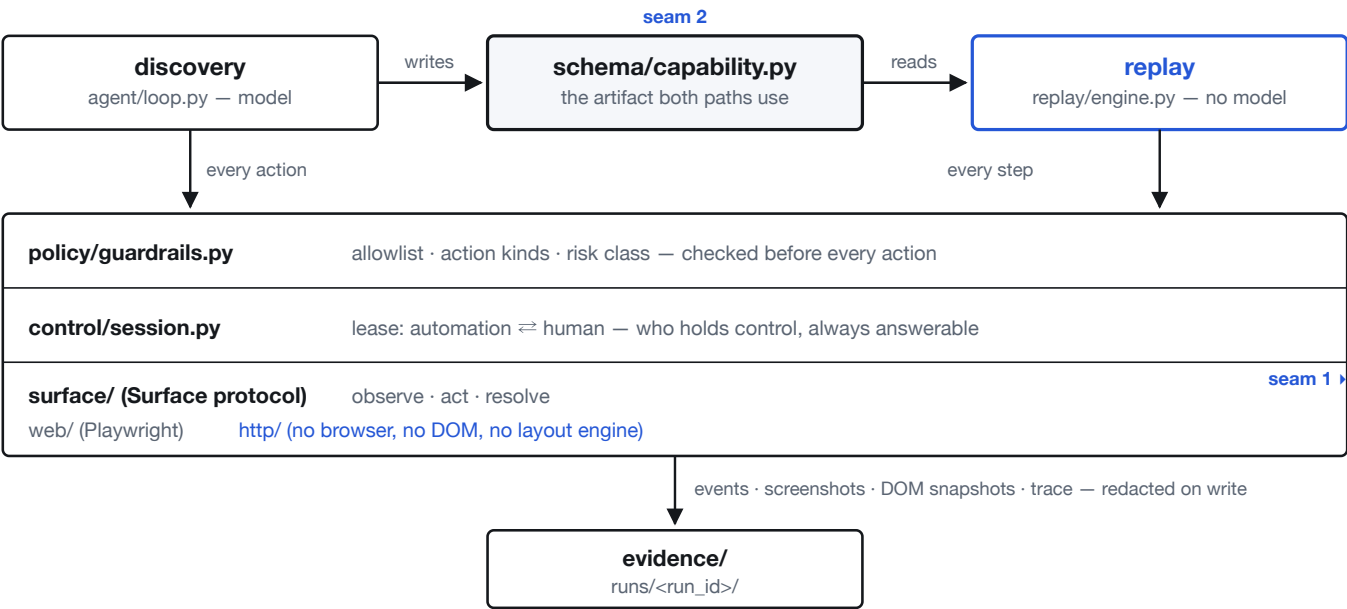


Figure 2 — The two seams are the whole design. Seam 1 is the surface: the only code that knows how a screen is reached. It is proven, not asserted, by a second implementation with no DOM and no JavaScript that replays the same reviewed artifact. **Seam 2** is the artifact: discovery writes it, a human reviews it, replay reads it — and nothing in it refers to the model transcript.

2 · Robustness — the locator ladder, measured

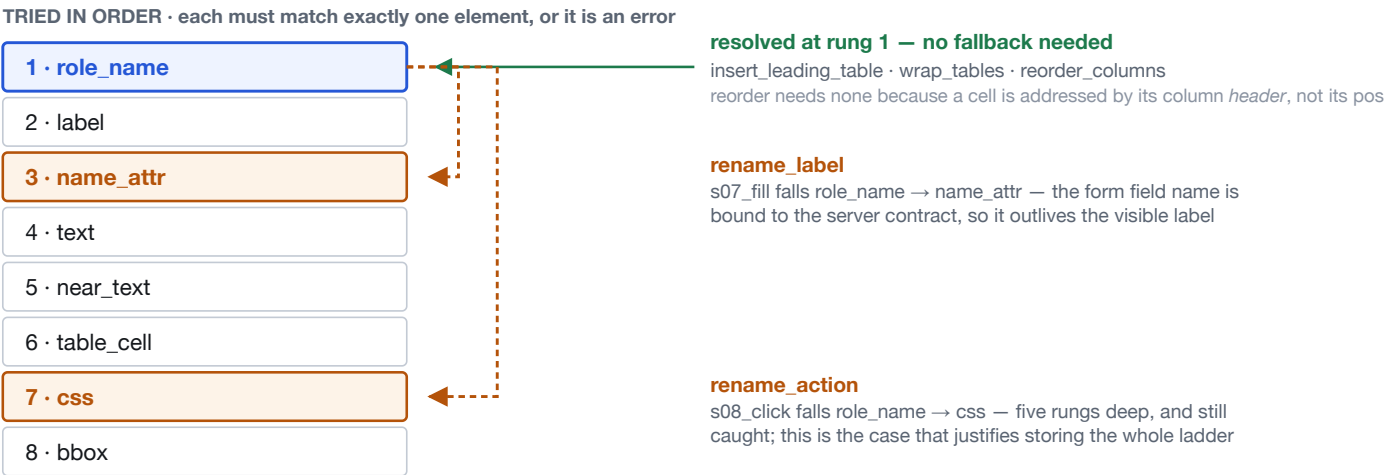


Figure 3 — Drift is a number, not a claim. glovebox drift rewrites the target's own rendered pages the way a redesign would and replays every approved capability against each, reporting *which rung caught the fall*. Both capabilities survive all five redesigns; the command exits non-zero if that ever changes, so a quietly narrowed ladder fails the build instead of surfacing in production.

3 · Escalation — a human takes the *same* live session

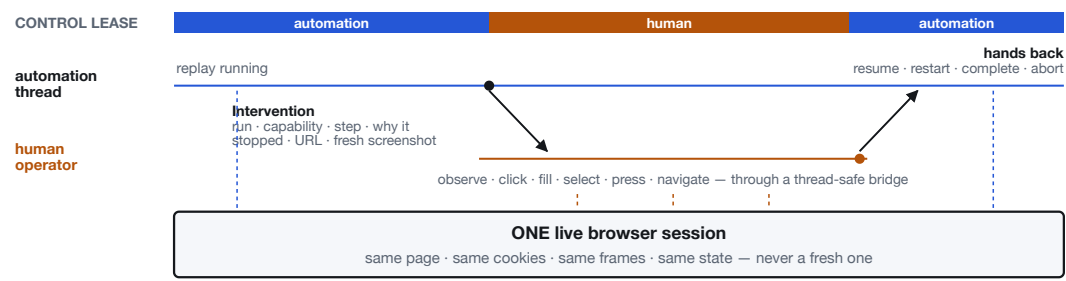


Figure 4 — The automation thread never exits; it stays alive *serving* the human. That is what makes this a handoff rather than a restart. Every human action is recorded with the recorder’s own multi-strategy target description, so the takeover is as auditable as the replay.

4 · Error taxonomy — classified in a fixed order after every step

OBSERVED	CLASSIFIED AS	RESULT
HTTP 5xx · “Application Error”	failure_signals	failed + screenshot and DOM snapshot of that moment
“No member record matched” · “Access denied”	outcomes	business_outcome — the caller branches, nothing broke
Session expired	recoveries → restart, max 2	success , steps marked recovered
Declared interstitial	recoveries → dismiss, retry	success
Undeclared modal · target missing	nothing matched → stuck	escalated if attended, else failed
Extracted text without the recorded shape	checkpoint	failed — expected vs observed, never a lucky guess
Bad parameter · draft artifact · disallowed URL	contract · policy	<i>refused before the application is touched</i>
Stopped from the console	cooperative cancel, between steps	failed / cancelled — retry the same call verbatim

Safety model

Exact-origin allowlist, denied path patterns, allowed action kinds — checked before every action in *both* paths. Each step is read | reversible | irreversible, rolled up; unattended replay of an irreversible capability is **refused**. “Confirm” over “block”, because opening a sub-account is what the product must do; over “allow”, because a wrongly recorded step would commit silently. Sensitive parameters are referenced *by name* and substituted at call time, so credentials never reach the transcript.

Stretch items built

- Approval gating (safety, robustness)
- Multi-run stability (safety, robustness)
- A bounded repair *proposal*, never applied (robustness)
- Cross-tenant overrides (generalization)
- The agent-facing catalog

Cut deliberately

Per-tenant session pool · fleet drift telemetry · Windows UIA surface · screenshot suppression · a typed frontend from the JSON Schema. Code generation was the one stretch item skipped.

The recorder guards, the drift evaluation and the browser-free surface were on no one’s list — each exists because a real run showed the argument it replaced was wrong.